

- 1.- R: Recordemos que Boolean es un tipo primitivo que solo tiene los valores true y false así que solo podemos hacer comparación, negación y un casteo al mismo tipo boolean.A, D y G
- 2.- R: A, B y D. Estamos manejando datos numéricos.
- 3.- R: B, C, D y F. Porque se manejan datos numéricos y debemos recordar que en algunos de estos se puede hacer el casteo en automático.
- 4.- R: B. Se realiza el análisis de escritorio.
- 5.- R: A y C. Las opciones A (+, *, %, --) y C (++ , (int), *) enumeran operadores ordenados según una precedencia creciente o constante.
- 6.- R: F. El código no compila en la línea 3 debido a la incompatibilidad de tipos de return
- 7.- R: D. Se evalúan los operadores tomando en cuenta el orden de precedencia y el comportamiento de estos.
- 8.- R: A.

Línea 4 (`int pig = (short)4;`): Se crea la variable `pig` con un valor inicial de `4`. El casteo a `short` es válido y se promociona automáticamente a `int`.

- **Línea 5 (`pig = pig++;`):** Esta línea utiliza un operador de post-incremento (`++`). En Java, el post-incremento devuelve primero el valor original y luego incrementa la variable. Por lo tanto, el lado derecho de la expresión evalúa a `4`. Aunque `pig` se incrementa momentáneamente a `5`, la asignación (`=`) sobrescribe inmediatamente ese valor con el `4` evaluado. Al final de esta línea, `pig` sigue siendo `4`.
- **Línea 6 (`long goat = (int)2;`):** Se crea la variable `goat` de tipo `long` con el valor de `2`.
- **Línea 7 (`goat -= 1.0;`):** Los operadores de asignación compuesta (como `-=`) realizan un casteo implícito en Java. La expresión es equivalente a `goat = (long)(goat - 1.0)`. La operación `2 - 1.0` da como resultado `1.0` (un tipo `double`), pero al ser truncado de vuelta a `long`, `goat` se convierte en `1`.
- **Línea 8 (`System.out.print(pig + " -" + goat);`):** Al concatenar las variables con el texto, se imprime el valor de `pig` (`4`), seguido del espacio y el guion (`-`), y finalmente el valor de `goat` (`1`).

9.- R: A, D y E. Analizar por medio de prueba de escritorio para ver el comportamiento de los operadores.

10.- R: G. Volver a estudiar los tipos de variables y como poder castear correctamente tomando en cuenta varias características.

11.- R: D. Realizar prueba de escritorio y analizar los operadores numéricos.

12.- R: D. El operador de post-incremento aumenta un valor y devuelve el valor original, mientras que el operador de pre-decremento disminuye un valor y devuelve el nuevo valor. Checar para que tipos aplica.

13.-R: F.

los operadores de bits/lógicos complejos (AND) y (XOR) tienen el mismo nivel de precedencia cuando actúan sobre booleanos, evaluándose de izquierda a derecha.

- Primero se evalúa la negación

- Luego el AND lógico

14.- R: B, E y G. Evaluación de operadores numéricos y tipos de variables

15.- D. operador condicional o ternario.

16.- R: B. En Java, no se puede asignar implícitamente un `long` a una variable `int` porque existe el riesgo de perder precisión; se requiere una conversión explícita a `int`.

17.- R: C y F. Prueba de escritorio.

18.- R: C. Símbolos de agrupación: Los paréntesis (), los corchetes [] y las llaves { } son símbolos de agrupación utilizados en matemáticas para modificar o alterar el orden estándar de las operaciones.

19.- R: B y F. Prueba de escritorio

20.- R: A, D y E. Precedencia de operadores y su clasificación.

21.- R: E. Prueba de escritorio y comportamiento de operadores.